

BUILDING WITH AI CODING AGENTS

The bottleneck didn't disappear. It moved.

5 problems nobody warns you about –
and the 8 practices that fixed them.

Swipe →

Writing code stopped being the slow part.

Knowing whether the code was **right** became the slow part.

Most teams have adopted the tools without reorganising around that shift – and it is the whole game.

Generation got cheap. **Verification didn't.**

Unreviewed AI code is just technical debt that arrived faster.

PART ONE

What actually goes wrong

Not the failures you read about.
The ones that cost me real hours.

PROBLEM 01

Confident wrongness

An agent recommended a data ingestion tool that was **retired in 2021**. Another suggested a function that had been **removed from the framework entirely**.

Both answers were fluent, well-structured and completely wrong.

*Training data has a cutoff.
Your production environment doesn't.*

PROBLEM 02

Review collapse

By the fifteenth diff of the day, I was skimming.

The agent produces faster than any human can genuinely review. So review quietly becomes **approval theatre** – and nobody notices the moment it happens.

Attention is now the scarce resource.

Not code.

PROBLEM 03

The 80% trap

Eighty percent of the feature in an hour. Then **four hours on the rest.**

Not because the last part was harder – because the agent made a structural choice early that I didn't notice, and everything after it was built on top.

The speed is real.

The finish line is further than it looks.

PROBLEM 04

Silent scope drift

I asked for one change.

I got that change, plus three **improvements** I hadn't asked for – and now had to review, understand and own forever.

Helpful and unrequested is still unrequested.

PROBLEM 05

Tests that prove nothing

Ask an agent for tests and it writes tests asserting **what the code does** – not what it should do.

The bug gets encoded into the test suite as expected behaviour.

*Green suite. Same bug.
Now with documentation.*

PART TWO

How I work now

Eight practices.
None of them are about prompting.

THE PRACTICES

1

I own the architecture. It owns the typing.

Delegate implementation, never design. **Every expensive mistake I've made was a design decision I let it take by default.**

2

Write the check before the code

A failing test. An assertion. A measurable property. **If you can't state what correct means, you can't delegate it.**

THE PRACTICES

3 Ground anything version-sensitive

Make it look things up rather than recall them. Versions, deprecations, API signatures, licence terms. **Verify, don't trust.**

4 One concern per change

Small enough that reviewing it is genuinely possible – **not theatre**. If you can't hold the diff in your head, it's too big.

THE PRACTICES

5

Make it prove the work, not claim it

Never accept “done”. **Ask for the command output, the passing test, the rendered result.** Evidence, not assurance.

6

Review the diff, not the explanation

The prose is always convincing – it was optimised to be. **The diff is the only thing that ships.**

THE PRACTICES

7

Start fresh when the thread degrades

Long sessions drift and constraints get forgotten. **A clean context beats arguing with a confused one.**

8

You define done. Never the agent.

It will tell you the work is complete. **That is a prediction about text, not a judgement about your system.**

THE UNDERLYING SHIFT

These tools move you from **author** to **reviewer**.

That's a genuine productivity gain. It's also a **different skill** – and one most of us have practised far less.

The engineers getting the most out of this aren't writing the cleverest prompts.

They got rigorous about verification.

What's broken for you that this list doesn't cover?

↓ **Tell me in the comments.**